Day 7 - Mini Project: Node.js CLI Task Tracker (Week 1 Review)

Day 7 — Mini Project: Node.js CLI Task Tracker (Week 1 Review)

Objectives

Combine everything from Days 1-6 into one working program:

- variables/types/equality (Day 1)
- control flow (Day 2)
- arrays and objects as your data model (Day 3)
- functions, scope, closures (Day 4)
- strings, template literals, JSON (Day 5)
- error handling and modules (Day 6)

This is intentionally less hand-held than the previous days — the point of a review project is to make the design decisions yourself, using what you've learned, and get stuck in the productive way.

What's actually new today: command-line arguments in Node

You haven't used these yet. Node gives you access to whatever was typed after your script's filename via a special array, `process.argv`:

```
console.log(process.argv);
```

If you ran this as `node tasks.js add "Buy milk"`, you'd see something like:

```
["/path/to/node", "/path/to/tasks.js", "add", "Buy milk"]
```

Notice `process.argv[0]` is always the path to the `node` program itself, and `process.argv[1]` is always the path to your script — the arguments you actually care about start at index **2**, not index 1 (a small but easy-to-forget difference from what you might expect, and a common early bug). A common first step in any script like this is to slice those off:

```
const args = process.argv.slice(2);   // ["add", "Buy milk"] -- just the REAL arguments, nothing els
```

The brief

Build a **command-line task tracker**, `tasks.js`, run repeatedly from the terminal, that manages a to-do list persisting between runs (saved as JSON — you covered `JSON.stringify`/`JSON.parse` on Day 5; today you'll additionally need to read and write actual files, using Node's built-in `fs` module, introduced just below).

Required features

1. `node tasks.js add "Buy milk"` — adds a new task, not yet complete
2. `node tasks.js list` — prints all tasks with an index number and a `[ ]`/`[x]` complete marker
3. `node tasks.js done 2` — marks task #2 as complete
4. `node tasks.js remove 2` — deletes task #2
5. Tasks persist in a file (e.g. `tasks.json`) between runs

Suggested data model

An array of objects:

```
[
    { description: "Buy milk", done: false },
    { description: "Walk dog", done: true },
]
```

Reading and writing files with Node's `fs` module

Node's standard library includes a module called `fs` ("file system") for reading and writing files — you haven't used this yet, so here's exactly what you need:

```
const fs = require("fs");

// writing (synchronously -- i.e., this line finishes completely before the next line runs)
fs.writeFileSync("tasks.json", JSON.stringify(tasks, null, 2));

// reading
const fileContents = fs.readFileSync("tasks.json", "utf-8");
const tasks = JSON.parse(fileContents);
```

The `Sync` in `writeFileSync`/`readFileSync` means "do this immediately, and don't move on to the next line until it's done" — the simplest way to work with files, and completely appropriate for a small script like this one. (Node also offers asynchronous, non-blocking versions of these — you'll understand exactly why that distinction matters once you cover asynchronous JavaScript on Day 10; for a small CLI tool like today's, the synchronous versions are simpler and entirely fine to use.)

You'll need to handle the case where `tasks.json` doesn't exist yet (the very first time the script ever runs) — `fs.readFileSync` throws an error if the file isn't there, so wrap it in `try`/`catch` (Day 6) and fall back to an empty array.

Suggested architecture (you decide the details)

- A function to load tasks from the JSON file (handling the file-not-existing-yet case without crashing)
- A function to save tasks back to the JSON file
- One function per command (`addTask`, `listTasks`, `completeTask`, `removeTask`)
- Parse `process.argv.slice(2)` by hand to figure out which command was requested and its arguments

Edge cases worth handling (this is where the real learning is)

- What happens if the user runs `list` before ever running `add`? (file doesn't exist yet)
- What happens if the user runs `done 99` but there are only 3 tasks? (index out of range — remember Day 3: JS won't crash on an out-of-range array index on its own, so you need an explicit check)
- What happens if the user runs `done abc` (not a number)?
- What happens if the user runs the script with no arguments, or an unrecognized command?

None of these should crash with an ugly, unhandled error — they should print a clear, human-readable message.

How to approach this (process, not just code)

1. Sketch the function signatures you'll need as comments first, before writing logic.
2. Build `add` and `list` first, test manually by running the script from the terminal, before adding `done`/`remove`.
3. Get the happy path fully working before handling edge cases.
4. Once it all works, deliberately try to break it (the edge cases above) and fix what breaks.

A note on scope

Don't reach for classes or anything from Week 2 — the whole point is proving you can build something real with *only* Week 1 material. You'll rebuild something similar with classes and TypeScript by Day 14.

What's provided

- `starter.js` — an empty scaffold with function signatures and comments to get you started (optional — you can also start from a blank file)
- `solution.js` — a full reference implementation. **Build your own first.**

Manual test checklist

Run these in order from your terminal in this folder:

```
node tasks.js add "Buy milk"
node tasks.js add "Walk dog"
node tasks.js list
node tasks.js done 1
node tasks.js list
node tasks.js remove 2
node tasks.js list
node tasks.js done 99
node tasks.js done abc
node tasks.js bogus-command
node tasks.js
```